CURSO PARA INTEGRADORES DO HUBSAÚDE · MÓDULO 3 DE 6

3 Montar registros FHIR

Antes de enviar qualquer informação ao HubSaúde, o seu sistema precisa escrevê-la no padrão FHIR e conferir se ela segue as regras. Neste módulo, você aprende a ler um registro, a encontrar os erros com o Validador e a corrigi-los antes do envio.

AO FINAL, VOCÊ SERÁ CAPAZ DE

- ✓ reconhecer a estrutura de um recurso FHIR em JSON;
- ✓ explicar o que são perfis e Guias de Implementação;
- ✓ validar um registro com o HubSaúde CLI e interpretar o resultado;
- ✓ corrigir erros de formato e de valores aceitos;
- ✓ consultar campos de um registro com FHIRPath.

COMO O MÓDULO ESTÁ ORGANIZADO

1. **Conceitos:** seções 3.1 a 3.5
2. **Prática guiada:** 7 passos no seu computador
3. **Verificação:** perguntas, glossário e respostas

VOCÊ VAI PRECISAR

- O HubSaúde CLI e o programa `jq`, instalados no módulo 2.
- A pasta `curso-hubsaude`, criada no módulo 2.
- Os arquivos `paciente.json`, `bundle.json` e `conferir.sh`: o conteúdo de cada um está neste material, e todos podem ser baixados na página Materiais do curso.

ASSISTA AOS VÍDEOS

Aponte a câmera do celular para o código.

PARTE 1
Anatomia de um registro FHIR
4 min 39 s, com narração e legendas

PARTE 2
Validar e corrigir um registro
6 min 30 s, com narração e legendas

DÚVIDAS E SUGESTÕES

Se algum passo ficar difícil, escreva para o suporte do HubSaúde, em `suporte@saude.go.gov.br`, com o assunto "Curso para integradores, módulo 3". A versão mais recente deste módulo, com os vídeos, está em `https://sesgo-ti.github.io/hubsaude/curso/registros/`.

Copiar de um PDF pode alterar aspas, hífens e quebras de linha. Para os comandos e arquivos, prefira a versão do módulo no portal ou os arquivos para baixar, em `https://sesgo-ti.github.io/hubsaude/curso/materiais/`.

PARTE 1

Conceitos

3.1 O que é um registro FHIR

O FHIR é um padrão internacional para a troca de informações de saúde: com ele, sistemas diferentes descrevem a mesma informação da mesma forma. No FHIR, cada unidade de informação é um **recurso**. Um paciente é um recurso do tipo Patient; um resultado de exame, do tipo Observation.

No HubSaúde, os recursos são escritos em JSON. Veja um paciente, com cada parte explicada ao lado:

paciente.json

```
1 {
2   "resourceType": "Patient", (1)
3   "name": [{"use": "official", "text": "Maria da Silva"}], (2)
4   "gender": "female", (3)
5   "birthDate": "1990-01-01" (4)
6 }
```

1. Informa o **tipo do recurso**. É por ele que quem recebe o registro sabe quais campos esperar.
2. O nome fica entre colchetes porque é uma **lista**: uma pessoa pode ter mais de um nome.
3. Aceita apenas valores de uma **lista fechada**, como você verá na seção 3.2.
4. Segue o **tipo de dado** data: ano, mês e dia, separados por hífen.

3.2 Tipos de dado e conjuntos de valores

A maior parte dos erros em registros FHIR vem de duas regras. Conhecê-las ajuda a montar registros corretos e a entender o que o Validador aponta.

Cada campo tem um tipo de dado

✗ RECUSADO

~~"birthDate": "01/01/1990"~~

✓ CORRETO

```
"birthDate": "1990-01-01"
```

Por quê? Uma data como 01/02 pode significar 1º de fevereiro ou 2 de janeiro, conforme o país. O formato ano, mês e dia tem uma única leitura.

Alguns campos aceitam apenas uma lista fechada

✗ RECUSADO

~~"gender": "F"~~

✓ CORRETO

```
"gender": "female"
```

Por quê? A lista fechada, chamada **conjunto de valores**, garante que todos os sistemas entendam o valor do mesmo modo. O campo `gender` usa o conjunto `AdministrativeGender`, que aceita `male`, `female`, `other` e `unknown`. Uma abreviação como `F` é recusada.

3.3 Perfis e Guias de Implementação

O padrão FHIR define a estrutura geral de cada recurso, para uso no mundo todo. As regras de cada uso específico ficam nos **perfis**, publicados nos Guias de Implementação do HubSaúde (`fhir.saude.go.gov.br`).

Padrão FHIR
A estrutura geral de cada recurso, igual no mundo todo.

Perfil
Acrescenta as regras de um uso: campos obrigatórios e valores permitidos. Publicado no Guia de Implementação.

O seu registro
Declara o perfil que segue no campo `meta.profile`.

▲ O Validador confere as regras do padrão e as do perfil declarado

O Validador confere o registro contra as duas camadas: as regras do padrão e as do perfil declarado. Por isso, um registro pode estar correto pelo padrão e, ainda assim, ser recusado por não seguir o perfil.

3.4 Bundle: registros que chegam juntos

Pense no pedido de um exame e nos seus resultados: são recursos diferentes, mas precisam chegar juntos. O **Bundle** agrupa esses recursos em um único registro.

bundle.json

```
{
  "resourceType": "Bundle",
  "type": "collection",
  "entry": [
    {
      "fullUrl": "urn:uuid:5b2c7c1e-6f1a-4c3e-9d2a-1f0e8a7b6c5d",
      "resource": {
        "resourceType": "Patient",
        "name": [{"use": "official", "text": "Maria da Silva"}],
        "gender": "female",
        "birthDate": "1990-01-01"
      }
    }
  ]
}
```

1. Um Bundle também é um recurso, com o seu próprio tipo.
2. A lista de itens. Cada item traz um recurso.
3. O **identificador** do item: um código único no formato `urn:uuid`. Outro recurso do mesmo Bundle aponta para este pelo `fullUrl`.
4. O recurso em si: aqui, o paciente da seção 3.1.

DICA

No ambiente oficial, os registros são enviados ao HubSaúde dentro de um Bundle, com o perfil indicado no Guia de Implementação correspondente. Os exercícios deste módulo começam com um recurso isolado e terminam com um Bundle simples.

3.5 Validar antes de enviar

No ambiente oficial, o HubSaúde aplica as regras completas dos Guias de Implementação e recusa o registro que não as cumpre. Veja a diferença entre descobrir um erro depois do envio e descobri-lo antes:

Erro descoberto depois do envio

Enviar → Task recusada → Ler o motivo → Corrigir → Enviar de novo

Erro descoberto pelo Validador

Validar no seu computador → Corrigir → Enviar uma vez

O Validador roda na sua máquina e confere o registro antes que ele saia. Durante o curso, use nele apenas dados fictícios; na integração real, o seu sistema também valida os registros verdadeiros antes de enviá-los.

Como ler o resultado

Para cada problema, o Validador escreve uma linha com três partes, sempre na mesma ordem:

```
error   Patient.birthDate   Não é uma data válida (01/01/1990)
```

Gravidade — o que fazer
Caminho — onde está o problema
Mensagem — o que está errado

O caminho começa pelo tipo do recurso e desce até o campo, com as partes separadas por ponto: `Patient.birthDate` é a data de nascimento do paciente. A gravidade diz o que fazer:

Gravidade	O que significa	O que fazer
`error`	O registro está fora das regras e será recusado.	Corrigir antes do envio.
`warning`	O registro é aceito, mas há uma recomendação.	Avaliar se vale atendê-la.
`information`	Apenas uma observação.	Nada.

O código de saída

Ao terminar, o programa devolve um **código de saída**: um número que scripts e outros programas conseguem ler sem interpretar o texto das mensagens. Assim, um script ou uma pipeline de integração contínua interrompe o envio quando o código é 1.

Código	Significado	Um script de envio…
0	Nenhum problema	segue
1	Há erros	interrompe o envio
2	Há apenas avisos	segue
3	Erro de uso, como uma opção inválida: o problema está no comando	interrompe, e o comando deve ser corrigido

EM RESUMO

- Cada registro é um recurso, escrito em JSON e identificado pelo `resourceType`.
- Cada campo segue um tipo de dado e, às vezes, um conjunto de valores, além das regras do perfil.
- O Bundle reúne os recursos que chegam juntos, cada um com o seu `fullUrl`.
- O Validador confere o registro antes do envio; cada problema traz gravidade, caminho e mensagem, e o código de saída permite automatizar a conferência.

PARTE 2

Prática guiada

Execute os comandos na pasta `curso-hubsaude`. Em cada passo, o quadro **Você digita** traz os comandos, e o quadro **Você vê**, o resultado esperado.

1 Criar um registro com erros

Salvar um paciente com dois erros propositais e lê-lo no terminal.

Crie o arquivo `paciente.json` com o conteúdo abaixo, ou baixe-o pronto.

paciente.json

```
{
  "resourceType": "Patient",
  "name": [{"use": "official", "text": "Maria da Silva"}],
  "gender": "F",
  "birthDate": "01/01/1990"
}
```

Para baixar: `https://sesgo-ti.github.io/hubsaude/curso/arquivos/paciente-com-erros.json`

DESAFIO

Antes de continuar, encontre os dois erros com base na seção 3.2.

O `jq` lê arquivos JSON no terminal. Escreva o nome do campo depois do ponto; em uma lista, indique a posição entre colchetes, a partir de zero:

Você digita	Você vê
`jq . paciente.json`	o registro inteiro, formatado
`jq .resourceType paciente.json`	"Patient"
`jq '.name[0].text' paciente.json`	"Maria da Silva"
`jq .birthDate paciente.json`	"01/01/1990"
`jq .gender paciente.json`	"F"

OBSERVE

O `jq` exibe `"F"` e `"01/01/1990"` sem apontar problema: para ele, os valores são apenas texto. Quem conhece as regras do FHIR e dos perfis é o Validador.

☐ **Passo concluído**

2 Validar o registro

Encontrar os erros com o Validador e ler o código de saída.

VOCÊ DIGITA

```
$ hubsaude validate paciente.json
$ echo $?
```

Na primeira execução, o Validador é baixado e iniciado, o que pode levar cerca de um minuto. Nas seguintes, a resposta é imediata.

VOCÊ VÊ

```
/home/usuario/curso-hubsaude/paciente.json      error
  error    Patient.gender     Unknown code 'http://hl7.org/fhir/administrative-gender#F'
  error    Patient.gender     O valor fornecido ('F') não foi encontrado no conjunto de valores
'AdministrativeGender' (…)
  error    Patient.birthDate  Não é uma data válida (01/01/1990)
  warning  Patient            A restrição falhou: dom-6 (definido em A resource should have narrative for
robust management) (Best Practice Recommendation)
1
```

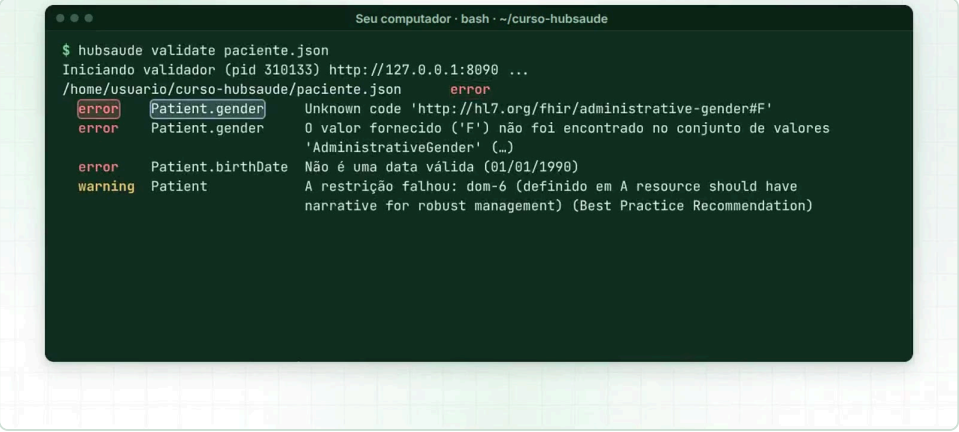


Figura 3.1. O mesmo resultado no terminal, com a primeira linha de problema em destaque. Quadro do vídeo "Validar e corrigir um registro".

OBSERVE

A primeira linha traz o resultado geral, `error`. O erro em `gender` aparece em duas linhas: uma diz que o código é desconhecido, e a outra, que ele não pertence ao conjunto de valores. O código 1 confirma: há erros.

☐ **Passo concluído**

3 Corrigir e validar novamente

Fazer as duas correções e confirmar que os erros desapareceram.

Abra o `paciente.json` no editor de texto e faça duas alterações. Depois, salve o arquivo.

NO EDITOR DE TEXTO: PACIENTE.JSON

linha 4	~~"gender": "F",~~	→ "gender": "female",
linha 5	~~"birthDate": "01/01/1990"~~	→ "birthDate": "1990-01-01"

VOCÊ DIGITA

```
$ hubsaude validate paciente.json
$ echo $?
```

VOCÊ VÊ

```
/home/usuario/curso-hubsaude/paciente.json      warning
  warning  Patient  A restrição falhou: dom-6 (definido em A resource should have narrative for robust
management) (Best Practice Recommendation)
2
```

OBSERVE

Os dois erros desapareceram. Resta o aviso `dom-6`, que recomenda um resumo em texto legível no campo `text`. Ele não impede a aceitação e pode ser mantido nos exercícios. O código 2 significa apenas avisos.

☐ **Passo concluído**

4 Consultar um campo com FHIRPath

Ler o valor de um campo sem abrir o arquivo.

FHIRPath é a linguagem do FHIR para apontar campos, com a mesma forma dos caminhos que o Validador mostra. Escreva a expressão sempre entre aspas simples.

VOCÊ DIGITA

```
$ hubsaude fhirpath 'Patient.birthDate' paciente.json
```

VOCÊ VÊ

```
1990-01-01
```

OBSERVE

O resultado confirma que a correção foi salva. Experimente também `'Patient.name.text'` e `'Patient.gender'`.

☐ **Passo concluído**

5 Agrupar o registro em um Bundle

Colocar o paciente dentro de um Bundle, validá-lo e percorrê-lo.

Crie o arquivo `bundle.json`, com o paciente já corrigido dentro da lista `entry`, e valide-o.

bundle.json

```
{
  "resourceType": "Bundle",
  "type": "collection",
  "entry": [
    {
      "fullUrl": "urn:uuid:5b2c7c1e-6f1a-4c3e-9d2a-1f0e8a7b6c5d",
      "resource": {
        "resourceType": "Patient",
        "name": [{"use": "official", "text": "Maria da Silva"}],
        "gender": "female",
        "birthDate": "1990-01-01"
      }
    }
  ]
}
```

Para baixar: `https://sesgo-ti.github.io/hubsaude/curso/arquivos/bundle.json`

VOCÊ DIGITA

```
$ hubsaude validate bundle.json
```

Quando há problemas dentro de um Bundle, o caminho indicado pelo Validador começa pelo item, como `Bundle.entry[0].resource`. O `jq` percorre o mesmo caminho:

Você digita	Você vê
`jq '.entry[0].fullUrl' bundle.json`	"urn:uuid:5b2c7c1e-6f1a-4c3e-9d2a-1f0e8a7b6c5d"
`jq '.entry[0].resource.resourceType' bundle.json`	"Patient"

☐ **Passo concluído**

6 Usar o código de saída em um script

Interromper um processo automaticamente quando o registro tiver erros.

Salve o script abaixo como `conferir.sh`. Ele valida o arquivo informado sem exibir as mensagens e decide pelo código de saída:

conferir.sh

```
#!/bin/bash
hubsaude validate "$1" > /dev/null (1)
codigo=$? (2)
if [ $codigo -ne 0 ] && [ $codigo -ne 2 ]; then (3)
  echo "Envio interrompido (código $codigo)"
  exit 1
fi
echo "Registro sem erros: pode seguir para o envio"
```

1. Valida o arquivo informado; `> /dev/null` esconde as mensagens.
2. Guarda o código de saída da validação.
3. Se o código não for 0 nem 2, interrompe o processo.

VOCÊ DIGITA

```
$ bash conferir.sh paciente.json
```

VOCÊ VÊ

```
Registro sem erros: pode seguir para o envio
```

Com um registro que contenha erros, o código seria 1, e o script exibiria `Envio interrompido (código 1)`. Veja também o código 3, de erro de uso, com uma opção que não existe:

VOCÊ DIGITA

```
$ hubsaude validate paciente.json --formato json
$ echo $?
```

VOCÊ VÊ

```
Erro: flag desconhecida: --formato
3
```

OBSERVE

O problema está no comando, e não no registro. Para obter o resultado em JSON, a opção correta é `-o json`.

☐ Passo concluído

7 Opcional: relatório e interface web

Conhecer as outras formas de ler o resultado da validação.

A opção `--report` gera um relatório em HTML, aberto no navegador, com a tabela de problemas e o OperationOutcome completo: o formato do FHIR para descrever problemas, o mesmo que o HubSaúde devolve quando recusa um registro.

VOCÊ DIGITA

```
$ hubsaude validate paciente.json --report
```

Sem o terminal, use a interface web do Validador, em `http://localhost:8090/index.html`: cole o conteúdo do arquivo e clique em **Validar**.

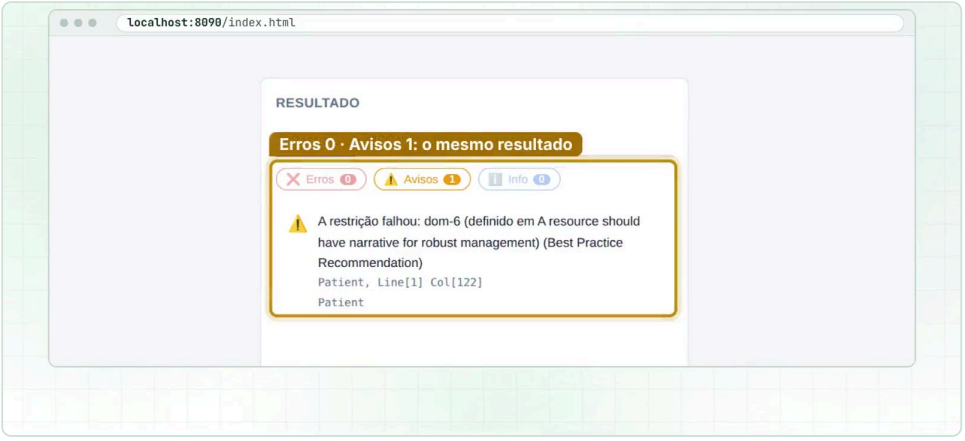


Figura 3.2. A interface web do Validador, com os mesmos problemas ao lado do registro. Quadro do vídeo "Validar e corrigir um registro".

☐ Passo concluído

PARTE 3

Verificação

Responda sem consultar o texto. As respostas estão no final do módulo.

1. O Validador apontou error em Patient.birthDate para o valor 29-02-2024. Qual é o problema?

2. Um registro foi validado e o programa terminou com o código 2. O registro pode ser enviado?

3. Qual a diferença entre o padrão FHIR e um perfil?

4. Como uma automação pode impedir o envio de um registro com erros?

Glossário do módulo

Bundle
Recurso que agrupa outros recursos que chegam juntos, cada um em um item da lista `entry`.

Código de saída
Número devolvido por um programa ao terminar, lido por scripts: no Validador, 0, 1, 2 ou 3.

Conjunto de valores
Lista fechada dos valores aceitos por um campo, como `AdministrativeGender`.

FHIRPath
Linguagem do FHIR para apontar campos, como `Patient.birthDate`.

fullUrl
Identificador de um item do Bundle, no formato `urn:uuid`.

Guia de Implementação
Publicação que reúne os perfis de um uso, com as regras que o HubSaúde aplica.

Perfil
Conjunto de regras de um uso específico, acrescentadas ao padrão FHIR.

Recurso
Unidade de informação do FHIR, como Patient ou Observation.

Validador
Ferramenta que confere um registro contra as regras do FHIR e dos perfis, antes do envio.

PRÓXIMO MÓDULO

4. Autenticar

Como o seu sistema prova quem é perante o HubSaúde, com um certificado digital, e obtém o token de acesso que acompanha cada envio.

Respostas da verificação

1. *O Validador apontou error em Patient.birthDate para o valor 29-02-2024. Qual é o problema?*

O formato. Datas no FHIR seguem o padrão `AAAA-MM-DD`; o valor correto seria `2024-02-29`.

2. *Um registro foi validado e o programa terminou com o código 2. O registro pode ser enviado?*

Sim. O código 2 indica que há apenas avisos, que não impedem a aceitação. Ainda assim, convém avaliar cada aviso.

3. *Qual a diferença entre o padrão FHIR e um perfil?*

O padrão FHIR define a estrutura geral de cada recurso. O perfil acrescenta regras de um uso específico, como campos obrigatórios e valores permitidos, e é publicado nos Guias de Implementação.

4. *Como uma automação pode impedir o envio de um registro com erros?*

Executando `hubsaude validate` antes do envio e interrompendo o processo quando o código de saída for 1. A opção `-o json` fornece o resultado em formato legível por programas.